Hitachi Solutions — Site Reliability Engineer

Comprehensive Interview Questions & Spoken Answers

Prepared from the supplied Hitachi Solutions Innovation Team SRE job description. The JD emphasizes hands-on Azure operations, AKS/Kubernetes, Terraform/IaC, CI/CD, observability, incident response, customer deployment troubleshooting, identity/secrets, data-platform awareness and documentation. ■filecite■turn0file0■L5-L9■

How to use this guide: For live interviews, answer the first 2–4 sentences naturally, then expand with the technical detail if the interviewer probes. For incident scenarios, use **Impact** → **Triage** → **Metrics/Logs** → **Isolate** → **Mitigate** → **Recover** → **RCA** → **Prevent**.

JD Area	Preparation Focus
Azure / AKS	Azure infrastructure, AKS, networking, Key Vault, RBAC, managed identity
IaC	Terraform state/drift, modules, plan/apply failures, Bicep/ARM/Helm
CI/CD	GitHub Actions, Azure DevOps, YAML, approvals, secrets, hotfixes
Kubernetes / GitOps	Troubleshooting, Helm, Argo CD/Flux, workloads, services, storage
SRE	SLO/SLI, error budget, incidents, RCA, MTTR, capacity, toil
Observability	Metrics/logs/traces, Prometheus/Grafana, Azure Monitor/App Insights
Data Platform	ADF, Databricks, Synapse, data deployment/reliability
Customer Support	Tenant configuration, deployment escalation, readiness, runbooks

1. Role & SRE Fundamentals

Q1. Tell me about yourself for this SRE role.

I'm an Azure-certified SRE/DevOps engineer with 11+ years of experience across cloud operations, Kubernetes, Terraform, CI/CD, observability and production support. In my recent role I supported Azure Synapse Analytics in a production SRE environment, handling P1/P2 incidents, monitoring, SLI/SLO work, reliability dashboards, alert optimization, CI/CD, platform health and capacity planning. I'm strongest when I can combine hands-on troubleshooting with automation and operational improvements. For this role, my focus would be keeping Azure-based SaaS and data-platform environments reliable, repeatable and well observed, while working closely with engineering, QA, security and delivery teams.

Q2. What is Site Reliability Engineering?

SRE applies software-engineering principles to operations. The objective is to make systems reliable, scalable and efficient while controlling operational toil. In practice I use SLIs and SLOs to define reliability, error budgets to balance reliability with delivery, automation to remove repetitive work, observability to detect and diagnose issues, and incident/RCA processes to continuously improve the platform.

Q3. What is the difference between SLA, SLO and SLI?

An SLI is the measured indicator, such as availability, latency or error rate. An SLO is the target for that indicator, for example 99.9% successful requests. An SLA is the externally agreed commitment, often with business consequences if it is missed. I normally define the SLI first, establish an SLO based on customer expectations and then ensure the operational process respects the relevant SLA.

Q4. What is an error budget?

For an availability SLO, the error budget is the amount of unreliability we can tolerate during the SLO window. For example, a 99.9% monthly availability target allows roughly 43.2 minutes of unavailability in a 30-day month. I use the budget to guide release risk: if reliability is consuming the budget too quickly, I would prioritize reliability work before additional risky changes.

Q5. How do you approach production reliability?

I start with measurable objectives: availability, latency, error rate, saturation and important business transactions. Then I establish observability, meaningful alerts, capacity thresholds, runbooks and incident procedures. I review recurring incidents and toil, automate repetitive work, and use post-incident findings to improve architecture, deployment safety and operational readiness.

Q6. What is toil in SRE?

Toil is repetitive, manual, automatable operational work that provides little enduring value. Examples are manually changing pipeline secrets, repeatedly restarting workloads, manually checking the same dashboards, or performing the same environment configuration steps. I reduce toil through Terraform, CI/CD automation, scripts, GitOps, standardized runbooks and self-service workflows.

Q7. How do you measure reliability?

I combine SLIs such as availability, request success rate, latency and freshness with operational metrics such as MTTR, incident frequency, change failure rate and alert quality. I also look at customer-impacting incidents and capacity trends. The key is to measure outcomes rather than simply counting infrastructure metrics.

2. Production Incident Response

Q8. A production deployment caused errors. What do you do?

First I establish impact: affected customers, regions, services and SLOs. Then I check recent changes, deployment history, application metrics, logs, traces and Kubernetes events. If the deployment is clearly causal, I mitigate quickly through rollback, traffic reduction or a safe configuration correction. After service restoration I validate health, document the timeline and perform RCA with preventive actions.

Q9. A P1 incident is reported. Walk me through your response.

I acknowledge and assess impact immediately, establish an incident channel and identify an incident owner. I avoid random changes and use a hypothesis-driven triage: recent changes, availability, latency, error rate, saturation, dependencies and logs. I prioritize mitigation over root-cause perfection. Once stable, I validate recovery against customer and SLO indicators, then capture timeline, root cause, contributing factors and corrective actions.

Q10. What is your incident-management pattern?

My practical pattern is Impact → Triage → Metrics/Logs → Isolate → Mitigate → Recover → RCA → Prevent. During an active outage I separate mitigation from investigation so we restore service quickly while preserving evidence for the later RCA.

Q11. How do you handle an issue you cannot resolve within your scope?

I first collect enough evidence to make escalation useful: impact, timeline, symptoms, logs, metrics, recent changes, reproduction details and actions already attempted. I identify the correct owner—application, networking, identity, security, database or cloud team—and escalate with a concise technical summary. I continue coordinating the incident rather than simply handing it off.

Q12. What should a good RCA contain?

It should describe customer impact, detection, timeline, root cause, contributing factors, why existing controls did not prevent or detect it, recovery actions and preventive actions. Preventive actions should have owners and measurable outcomes. I avoid blaming individuals and focus on system and process improvements.

Q13. How do you reduce MTTR?

Improve detection quality, reduce alert noise, maintain accurate dashboards and runbooks, standardize rollback procedures, automate diagnostics and recovery, improve service ownership, and practice incident response. I also use post-incident data to identify the largest recurring sources of recovery time.

3. Azure Core & Architecture

Q14. What Azure services in this JD are important for an SRE?

AKS for Kubernetes workloads; Key Vault for secrets; Storage Accounts for durable storage; Data Factory for data movement/orchestration; Databricks for data processing; Azure Monitor/Application Insights for observability; VNets, private endpoints and private DNS for network isolation; RBAC and managed identities for access control; and Azure DevOps/GitHub Actions for delivery automation.

Q15. Explain Azure RBAC.

Azure RBAC controls who can perform which actions on which Azure resources. The core model is security principal plus role plus scope. I follow least privilege, prefer managed identities or workload identities over long-lived credentials, and scope access as narrowly as practical at management-group, subscription, resource-group or resource level.

Q16. What is Azure Key Vault and how do you use it in CI/CD?

Key Vault stores secrets, keys and certificates securely. In CI/CD I prefer workload identity or managed identity where supported, grant only required permissions, retrieve secrets at runtime rather than hardcoding them, and audit access. Pipeline logs must never expose secret values.

Q17. Managed identity vs service principal?

A service principal is an application identity; it commonly requires credential management unless certificate or federated authentication is used. Managed identity is an Azure-managed identity associated with a resource. Where possible I prefer managed identity or workload identity because it reduces secret-management overhead and credential exposure.

Q18. What is a private endpoint?

A private endpoint maps an Azure service to a private IP in a VNet using Private Link. It keeps traffic on private networking rather than requiring public access to the service. Correct DNS configuration is critical because clients must resolve the service name to the private endpoint address.

Q19. Private endpoint works but application cannot connect. What do you check?

I check DNS resolution from the application subnet, private DNS zone linkage, route tables, NSGs/firewalls, the private endpoint connection state, service-side authorization, TLS and the application's configured hostname. I also test from the same network context as the workload rather than from my laptop.

Q20. What Azure networking concepts should an SRE know?

VNets and subnets, NSGs, route tables, peering, private endpoints, private DNS zones, NAT, firewalls, load balancers, ingress and service endpoints. For troubleshooting I determine whether the failure is DNS, routing, network policy, identity, service availability or application-level connectivity.

4. AKS & Kubernetes

Q21. Explain the Kubernetes architecture.

The control plane manages desired state through components such as the API server, scheduler and controllers, while worker nodes run workloads through the kubelet and container runtime. In AKS, Azure manages the control-plane experience while customers operate and configure the worker-side workloads and cluster resources within the service model.

Q22. A pod is in CrashLoopBackOff. What do you check?

I check kubectl get pods, describe the pod, current and previous container logs, exit codes, events, probes, environment variables, ConfigMaps and Secrets. Then I check resource limits, dependencies and recent deployment changes. CrashLoopBackOff is a symptom of repeated container failure, not itself the root cause.

Q23. Pod is Pending. How do you troubleshoot?

I inspect kubectl describe pod and events first. I check scheduling constraints, node capacity, taints/tolerations, affinity, resource requests, PVC binding and topology constraints. If it is resource-related, I compare requested CPU/memory with allocatable node capacity and cluster autoscaling behavior.

Q24. ImagePullBackOff?

I check pod events for the exact pull error, image name/tag, registry availability, imagePullSecrets, workload identity or registry permissions, network connectivity and whether the tag exists. I also verify the node can reach the registry and that the image architecture is compatible.

Q25. What causes OOMKilled?

The container exceeded its memory limit and the kernel/container runtime killed it. I check current and historical memory usage, requests and limits, workload behavior, recent changes and whether the limit is appropriate. I avoid simply increasing limits without understanding whether there is a memory leak or abnormal workload pattern.

Q26. How do Kubernetes readiness and liveness probes differ?

Readiness determines whether a pod should receive traffic. Liveness determines whether the container should be restarted. A bad liveness probe can create restart loops, while a bad readiness probe can remove healthy capacity from service. I design probes around meaningful application health rather than merely checking that a process exists.

Q27. How do you troubleshoot a Kubernetes service that is unreachable?

I verify the Service selector and endpoints, pod readiness, service port and targetPort, DNS, NetworkPolicy, ingress/load balancer configuration and application listening address. I test connectivity from inside the cluster to separate cluster networking problems from external ingress problems.

Q28. What is Helm?

Helm is a Kubernetes package and templating mechanism. Charts package Kubernetes resources and values allow environment-specific configuration. I use Helm with versioned charts, controlled values, validation and rollback capability, while keeping secrets out of source control.

Q29. What is GitOps?

GitOps treats Git as the declarative source of truth for desired environment state. A controller such as Argo CD or Flux reconciles the cluster to that state. This improves auditability, repeatability and rollback, and reduces manual cluster changes.

Q30. How would you troubleshoot an Argo CD/Flux deployment not syncing?

I check controller health and logs, repository connectivity and authentication, manifest rendering, target cluster connectivity, namespace/RBAC permissions, sync errors and resource health. I compare desired Git state with live state and inspect the specific resource that failed rather than treating the whole application as one failure.

5. Terraform & Infrastructure as Code

Q31. What is Terraform state?

State maps Terraform configuration to real infrastructure resources. Terraform uses it to determine what exists and what changes are required. In team environments I use remote state with locking and controlled access, protect sensitive state, and avoid manual state manipulation unless there is a justified recovery procedure.

Q32. What is Terraform drift?

Drift occurs when real infrastructure changes outside Terraform and no longer matches the declared configuration/state expectations. I detect it through plan, determine whether the change was intentional, then either codify the desired change or safely reconcile the resource.

Q33. Terraform plan shows an unexpected replacement. What do you do?

I inspect the plan and identify which attribute forces replacement. Then I compare the configuration, provider behavior, state and actual resource. If replacement is unsafe, I do not blindly apply it; I investigate lifecycle settings, dependencies, imports or configuration changes and assess production impact first.

Q34. Terraform apply failed halfway. How do you recover?

First I inspect the error and current state rather than immediately rerunning everything. Terraform often records successfully created resources in state. I validate the actual infrastructure, refresh/plan as appropriate, fix the root cause and then apply again. I also check for partial resources, dependency issues and provider/API limits.

Q35. How do you structure Terraform for multiple environments?

I use reusable modules with environment-specific variables and controlled state separation. Depending on the organization, separate state per environment or workspace can be used. I keep common logic in modules, avoid copy-paste, enforce formatting/validation/plan checks in CI and require review before production changes.

Q36. Terraform state is locked. What do you do?

I determine whether another legitimate operation is running before considering force-unlock. I inspect the lock owner and pipeline/job history. Force-unlock is a recovery action only after confirming the original operation is no longer active, because breaking a valid lock can create concurrent state corruption risk.

6. CI/CD, GitHub Actions & Azure DevOps

Q37. Design a reliable CI/CD pipeline for this role.

I would separate build, test, security validation, artifact creation, infrastructure validation and deployment stages. I would use immutable artifacts, environment approvals, controlled variables/secrets, deployment health checks and rollback procedures. Production deployment should be auditable and gated by readiness criteria.

Q38. GitHub Actions vs Azure DevOps pipelines?

Both support YAML-based CI/CD, secrets, approvals and reusable automation. The choice depends on the organization's source-control, governance and Azure integration model. I focus on pipeline design principles rather than treating the tool as the reliability boundary.

Q39. How do you secure pipeline secrets?

Use Key Vault or the CI/CD platform's secure secret store, prefer federated/workload identity where possible, restrict access, avoid printing secrets, rotate credentials, separate environment permissions and audit usage. Secrets should never be committed to Git or embedded in container images.

Q40. How would you implement deployment approvals?

I define environment protection rules so production requires appropriate review and checks. The pipeline should show artifact/version, change scope, test results, deployment plan and rollback procedure. Approvals should be auditable and tied to risk rather than becoming an automatic manual checkbox.

Q41. Explain a safe hotfix process.

Confirm impact and scope, identify the smallest safe change, create a controlled branch/change, run focused validation, obtain required approval, deploy to the appropriate environment, monitor key health indicators and keep rollback ready. Afterward merge the fix back into the normal development branch so the issue does not reappear.

Q42. What Git branching strategy do you prefer?

I adapt to the team's delivery model. For a continuously delivered product, trunk-based development with short-lived branches can reduce merge complexity. For release-oriented environments, a structured release/hotfix approach may be appropriate. Whatever the strategy, I enforce pull requests, review, automated validation and traceable release tags.

7. Observability & Monitoring

Q43. What are the three pillars of observability?

Metrics, logs and traces. Metrics show system behavior and trends, logs provide event-level context, and traces show request flow across distributed services. Modern observability also includes structured events, profiling and useful business indicators where applicable.

Q44. How would you design observability for a SaaS platform?

Start with critical user journeys and SLOs. Instrument request rate, errors, latency and saturation; collect structured application and infrastructure logs; add distributed traces for cross-service paths; build service and dependency dashboards; and create alerts tied to symptoms and SLO impact rather than every low-level metric.

Q45. How do you reduce alert fatigue?

Remove duplicate and non-actionable alerts, aggregate related symptoms, use severity based on customer impact, tune thresholds using historical behavior, add alert suppression during known maintenance and route alerts to the correct owner. I measure alert volume and actionable-alert percentage after changes.

Q46. Prometheus is not scraping a target. What do you check?

I check target discovery, ServiceMonitor/PodMonitor configuration if applicable, labels/selectors, endpoint path and port, network connectivity, TLS/authentication, Prometheus configuration and Prometheus logs. Then I inspect the target status in Prometheus to see the exact scrape error.

Q47. Grafana dashboard is showing no data.

I first determine whether the problem is the dashboard, Grafana datasource or underlying telemetry. I test the datasource directly, inspect the query and time range, verify Prometheus/Log Analytics data exists, check variable filters and permissions, and compare with a known-good query.

Q48. What makes a good production dashboard?

It should answer: Is the service healthy? Is the customer impacted? What changed? Where is the bottleneck? I include SLO status, traffic, errors, latency, saturation, dependency health, capacity and recent deployment indicators. I avoid filling dashboards with metrics that do not support an operational decision.

8. Azure Data Platform: ADF, Databricks, Synapse

Q49. What should an SRE know about Azure Data Factory?

ADF is commonly used for data integration and orchestration. From an SRE perspective I care about pipeline success/failure, duration, retries, integration runtime health, dependency failures, credentials, network connectivity, alerts and operational runbooks.

Q50. What should an SRE monitor in Databricks?

Job success/failure, duration, cluster health, resource utilization, driver/executor issues, Spark failures, data freshness, dependency health, secrets and access, workflow retries and cost/capacity trends. For customer environments I also

validate configuration promotion and environment-specific settings.

Q51. What is important when deploying data-platform components across environments?

Separate environment configuration from immutable deployment artifacts, control secrets and identities, validate network access and data permissions, use repeatable automation, test dependencies and define rollback or recovery procedures. Data changes require extra care because rollback may not be equivalent to rolling back application code.

Q52. How would you troubleshoot a Synapse/Databricks job suddenly becoming slow?

I compare current behavior with the baseline, check recent code/config/data-volume changes, cluster/resource utilization, Spark stages and skew, storage/network latency, external dependencies and concurrency. I separate infrastructure saturation from workload inefficiency before changing compute size.

9. Identity, Security & Secrets

Q53. A deployment fails with authorization denied. What do you check?

I identify the calling identity, target resource and exact denied action. Then I check Azure RBAC role assignments and scope, Key Vault permissions, service principal or managed identity configuration, token/credential validity, network restrictions and whether the pipeline is using the expected identity.

Q54. What is least privilege?

Give an identity only the permissions required for its task and no broader access. I apply it at the narrowest practical scope and periodically review access. For production automation, I prefer dedicated identities with clear ownership and controlled permissions.

Q55. How would you rotate a production secret safely?

First identify consumers and whether they support dual credentials. Create the new secret, update the consumer through controlled automation, validate connectivity, then revoke the old credential after the transition. I avoid changing both sides simultaneously without a recovery path.

Q56. How do you prevent secrets from leaking into logs?

Use secret-store references and masking features, avoid echoing environment variables, carefully review shell commands, sanitize exception output and prevent debug logging of authorization headers or connection strings. I also scan repositories and CI artifacts for accidental secret exposure.

10. Linux, Networking & Scripting

Q57. High CPU on a Linux server—what do you do?

I confirm whether the CPU is truly saturated using top, uptime, mpstat or similar tools, identify the process consuming CPU, inspect load average and I/O wait, compare with historical behavior and check recent changes. Then I determine whether to restart, scale, optimize or investigate a runaway process, based on production impact.

Q58. Disk is 95% full. What is your approach?

I identify filesystem usage with df -h, then locate large directories/files with du and targeted find commands. I check logs, temporary files, container layers and deleted-but-open files. I clean only safe data according to retention policy, then address the underlying cause through rotation, retention, capacity or application fixes.

Q59. How would you write a script to delete logs older than 30 days?

I would first identify the exact log path and retention requirement, use find with an age condition, perform a dry run where possible, log the action, and then delete only files matching the intended pattern. In production I would prefer a managed log-rotation or retention mechanism when available rather than an unsafe recurring deletion script.

Q60. A service is down. What Linux checks do you perform?

Check systemctl status, service logs, process state, listening ports with ss, recent configuration changes, dependencies, disk/memory/CPU and permissions. I verify the service after restart and monitor whether it remains healthy instead of assuming restart equals resolution.

Q61. How do you troubleshoot DNS?

Start with nslookup/dig from the same network location as the affected workload. Check authoritative records, private DNS zones, TTL, split-horizon behavior, search domains, DNS reachability and whether the resolved IP is correct. In Azure private endpoint scenarios, I specifically verify private DNS zone linkage and records.

Q62. How do you troubleshoot network connectivity?

I move layer by layer: DNS resolution, route, security controls, TCP connectivity, TLS and application protocol. I use tools such as dig/nslookup, curl, nc, traceroute where appropriate, and cloud-native network diagnostics. I compare source and destination context because network behavior can differ by subnet or identity.

11. Customer Deployment & Escalation Scenarios

Q63. A customer deployment fails but your environment works. What do you investigate?

I compare the customer's environment against the known-good baseline: subscription/tenant, region, resource names, RBAC, Key Vault, networking, private DNS, configuration values, versions, quotas and dependencies. I identify the first failing step and reproduce it where possible. I document the difference and either fix within scope or route a focused escalation.

Q64. Customer says the deployment is stuck. How do you communicate?

I give a concise status: what is affected, what we have confirmed, what we are checking, current mitigation, and the next update point. I avoid speculative root causes. Internally I keep technical detail high; externally I focus on impact, actions and expected next steps.

Q65. How do you create a good deployment runbook?

Include prerequisites, access requirements, environment variables, dependencies, exact deployment steps, validation checks, rollback, common failures, escalation contacts and troubleshooting commands. I keep it version-controlled and test it during releases rather than treating documentation as static text.

Q66. What would you check before declaring an environment production-ready?

Infrastructure health, networking, identity, secrets, monitoring, alert routing, logging, dashboards, backup/recovery where applicable, scaling, deployment/rollback, dependencies, security controls, runbooks, access and smoke tests. I also confirm that ownership and escalation paths are clear.

12. Scenario-Based Senior SRE Questions

Q67. AKS pods are healthy but users receive 502 errors. What do you do?

I work from the request path: external endpoint/ingress, load balancer, ingress controller, service, endpoints, pod readiness and application listener. I inspect ingress/controller logs and metrics, service configuration and recent changes. Healthy pods do not prove the network path is healthy, so I isolate each hop.

Q68. After a release, latency increases but error rate remains normal. How do you investigate?

I compare latency percentiles before and after release, identify affected endpoints, inspect traces for slow dependencies, check CPU/memory/network and database or external-service latency, and correlate with deployment changes. I avoid relying only on averages because tail latency often exposes the customer impact.

Q69. Production is intermittently failing and no single service is obviously down.

I look for correlated signals across services, regions and dependencies. I check traces, error codes, DNS, network health, throttling, resource saturation and recent configuration changes. Intermittent issues often require time correlation and request-level evidence rather than a simple up/down check.

Q70. Terraform pipeline is passing in QA but failing in production.

I compare identities, subscriptions, provider versions, state, variables, resource existence, policy assignments, quotas and network access. I run a production plan in the same identity/context and inspect the first meaningful error. Environment parity is important, but production often has additional policy and security controls.

Q71. GitOps repeatedly reverts a manual Kubernetes change. Is that a problem?

It is expected behavior if Git is the declared source of truth. The correct approach is to change the desired state in Git through the normal review process. Manual cluster changes should be reserved for controlled emergency mitigation and then reconciled back into Git afterward.

Q72. A secret suddenly stops working after rotation.

I verify which secret version the workload is consuming, whether the application refreshed the value, identity permissions, Key Vault access, configuration references and rollout behavior. If dual credentials are supported, I use overlap during rotation. I restore service first, then determine why the rotation process failed.

Q73. An alert fires every night but customers are not impacted.

I investigate the underlying signal and historical baseline, determine whether it represents expected batch behavior, and then tune the alert or schedule-based suppression if appropriate. I would not simply silence it without confirming that it cannot indicate a real failure.

Q74. You are asked to make a risky production change urgently. What do you do?

I clarify customer impact and business urgency, assess rollback, validate the minimum safe change, ensure the required authorization is present and capture the change. If the risk is unacceptable or security-sensitive, I escalate rather than bypassing governance. Urgency changes response speed; it does not eliminate production controls.

13. Behavioral & Collaboration Questions

Q75. Why do you want this SRE role?

The role aligns closely with my hands-on background in Azure, Kubernetes, Terraform, CI/CD, observability and production support. I'm particularly interested in the combination of customer-facing SaaS/data-platform operations and automation, where reliability improvements can be directly tied to deployment quality and customer outcomes.

Q76. How do you work with developers during an incident?

I focus on shared facts and customer impact. I bring metrics, logs, traces and deployment history, avoid assigning blame, and agree on the fastest safe mitigation. After recovery, I collaborate on the root cause and preventive engineering changes.

Q77. How do you prioritize multiple production issues?

Customer impact, severity, SLO risk, number of affected users, security implications and business criticality drive priority. I stabilize the highest-impact issue first while ensuring other incidents are acknowledged and owned. I communicate trade-offs clearly.

Q78. Tell me about a difficult production problem you solved.

A strong answer should follow Impact → Triage → Evidence → Isolation → Mitigation → Recovery → RCA → Prevention. For my background, I would use a real Azure/Synapse or Kubernetes incident and quantify the impact, explain the evidence used to isolate the issue, describe the recovery, and finish with automation or monitoring improvements.

Q79. How do you handle after-hours production support?

I prepare before the window: validate runbooks, access, dashboards, rollback and communication channels. During the change I monitor the relevant SLO and deployment signals closely. After completion I document the outcome and any follow-up actions so the process becomes easier and safer next time.

Q80. What is your approach to Agile ceremonies?

I use ceremonies to surface operational risk, dependencies and readiness early. In planning I identify deployment and infrastructure work; in stand-ups I highlight blockers; in retrospectives I turn incidents and delivery problems into actionable improvements; and during backlog review I help prioritize reliability and automation work.

14. Rapid-Fire Questions

Q81. What is a Kubernetes Deployment?

A declarative controller that manages ReplicaSets and provides rollout and rollback behavior for stateless workloads.

Q82. What is a ConfigMap?

A Kubernetes object for non-sensitive configuration. Secrets should be used for sensitive values, with appropriate external secret management where required.

Q83. What is a Kubernetes Secret?

A Kubernetes object for sensitive configuration data. It still requires access controls and encryption/secret-store integration; simply calling something a Secret does not make it secure.

Q84. What is a namespace?

A logical isolation boundary for Kubernetes resources within a cluster, commonly used for organization, access control and environment/application separation.

Q85. What is a Service?

A stable network abstraction that routes traffic to a set of pods selected by labels.

Q86. What is an Ingress?

A Kubernetes API abstraction for HTTP/HTTPS routing into services, implemented by an ingress controller.

Q87. What is a StatefulSet?

A controller for stateful workloads requiring stable identities and ordered deployment/scaling semantics.

Q88. What is a DaemonSet?

A controller that ensures a pod runs on eligible nodes, commonly used for node-level agents.

Q89. What is a PVC?

A PersistentVolumeClaim is a request for persistent storage from a Kubernetes workload.

Q90. What is blue-green deployment?

Maintaining two environments/versions and switching traffic between them to reduce deployment risk and enable rapid rollback.

Q91. What is canary deployment?

Releasing a change to a small subset of traffic/users first, validating telemetry, then expanding progressively.

Q92. What is immutable infrastructure?

Replacing infrastructure instances with new versions rather than modifying them in place, improving consistency and repeatability.

Q93. What is idempotency?

An operation is idempotent when repeating it produces the same intended state rather than accumulating unintended effects.

Q94. What is change failure rate?

The percentage of production changes that require remediation, rollback, hotfix or otherwise cause failure/incident impact.

Q95. What is MTTR?

Mean time to restore/recover service after an incident; the exact organizational definition should be agreed and measured consistently.

Q96. What is MTBF?

Mean time between failures, a reliability metric describing the average operating time between failures.

Q97. What is a runbook?

A documented operational procedure for performing a task or troubleshooting a known failure mode.

Q98. What is a postmortem?

A structured review of an incident covering impact, timeline, causes, recovery and prevention.

Q99. What is Git rebase?

A Git operation that reapplies commits on top of another base, creating a cleaner linear history when used appropriately.

Q100. What is a pull request?

A mechanism for proposing, reviewing, validating and merging source changes under repository governance.

15. High-Probability Questions for This JD

Q101. Why are you a fit for this role?

My experience maps directly to the core requirements: Azure production operations, Kubernetes/AKS concepts, Terraform, CI/CD, observability, incident response, SLI/SLOs, customer-impact troubleshooting and automation. I also have hands-on experience with Azure Synapse and data-platform operations, which is relevant to the data-platform side of this position.

Q102. Explain a complete AKS production deployment from Git commit to production.

A developer submits a pull request; CI runs tests, linting and security checks; an immutable image is built and pushed to a registry; infrastructure/configuration is validated; the deployment manifest or Helm values are promoted through GitOps or the approved pipeline; production approval gates are evaluated; the workload rolls out; readiness and application health are validated; telemetry is monitored against SLOs; and rollback is available if health degrades.

Q103. Design an Azure SaaS environment for reliability.

Use isolated VNets/subnets and private connectivity where required, AKS with appropriate scaling and availability, managed identity/RBAC, Key Vault, durable storage, centralized monitoring/logging, controlled CI/CD, Terraform-based provisioning, backup/recovery controls and documented runbooks. Define SLOs and build dashboards and alerts around customer journeys.

Q104. How would you automate customer environment deployment?

Create reusable Terraform modules and versioned application artifacts, parameterize customer/environment configuration, automate identity and Key Vault setup, validate networking and prerequisites, deploy through CI/CD or GitOps, run post-deployment smoke tests and publish a deployment record. The goal is a repeatable workflow rather than customer-specific manual steps.

Q105. What would you improve first if you joined the team?

I would first understand the existing SLOs, incident history, deployment process, alert quality, infrastructure-as-code maturity and customer escalation patterns. I would then identify the highest recurring operational pain points and prioritize improvements based on customer impact, frequency and engineering effort rather than making assumptions before observing the environment.

16. Questions You Can Ask the Interviewer

Q106. What are the primary SLOs for the product and how are they measured?

Shows focus on reliability outcomes and operational measurement.

Q107. How are AKS deployments currently managed—Azure DevOps, GitHub Actions, Argo CD, Flux, or a combination?

Clarifies the actual delivery and GitOps model.

Q108. How is Terraform state organized across customer environments?

Helps understand environment isolation, governance and operational risk.

Q109. What are the most common production incidents this SRE team handles?

Reveals the real operational workload behind the JD.

Q110. How are customer deployment escalations triaged and owned?

Clarifies support boundaries and collaboration with Web Support, engineering and delivery.

Q111. What does the on-call/24x7 model look like?

Clarifies support expectations, escalation and after-hours deployment requirements.

Q112. What observability stack is currently standardized?

Connects the role to the JD's metrics/logs/traces/dashboard requirements.

Q113. How do you measure SRE success after the first 6 months?

Surfaces the team's reliability, delivery and automation expectations.

Final 24-Hour Interview Checklist

1. Prepare a 60-second introduction mapped to Azure + SRE + Kubernetes + Terraform + CI/CD + observability.

2. Prepare 2 real production incidents using Impact → Triage → Evidence → Mitigation → Recovery → RCA → Prevention.

3. Practice AKS troubleshooting: CrashLoopBackOff, ImagePullBackOff, Pending, OOMKilled, Service/Ingress and DNS.

4. Practice Terraform: state, drift, locking, failed apply, unexpected replacement and environment separation.

5. Practice CI/CD: GitHub Actions/Azure DevOps, approvals, secret management, hotfix and rollback.

6. Practice Azure identity: RBAC, Key Vault, managed identity/service principal, private endpoint/private DNS.

7. Review Prometheus/Grafana and Azure Monitor/Application Insights troubleshooting.

8. Review ADF, Databricks and Synapse operational scenarios.

9. Prepare one customer deployment escalation example and one automation/toil-reduction example.

10. Know your metrics: availability, latency percentiles, error rate, saturation, SLO, error budget, MTTR and change failure rate.